

JAVA INTERVIEW QUESTION BANK

Topic-wise Questions + Interview-Ready Answers

Reorganized from the provided Question Bank

Format: concise explanation • comparison tables • code examples • interview tips

Contents

- 1. Java Fundamentals & Object Model
- 2. OOP, Interfaces, Immutability & Design
- 3. Strings
- 4. Collections, Hashing & Generics
- 5. Functional Interfaces & Streams
- 6. Exceptions & Resource Management
- 7. Serialization, Copying & Object Lifecycle
- 8. Multithreading & Concurrency
- 9. JVM Memory Model, OOM & Garbage Collection
- 10. Modern Java: 8 / 11 / 17 / 21 / 25
- 11. Annotations
- 12. Testing, Mockito & Dependency Injection
- 13. Reactive Programming
- 14. Stream API Coding Questions

NOTE: Duplicate questions from the original mixed list are consolidated under the correct topic so you do not study the same concept twice. Every unique question/topic from the source is represented.

1. Java Fundamentals & Object Model

Q1. Why is the main method static?

In the traditional Java launcher model, the JVM must be able to invoke the entry point without first creating an object of the class. Therefore the standard entry point is static: `public static void main(String[] args)`.

- public: launcher can access it.
- static: no instance is required.
- void: no return value is expected by the JVM.
- String[] args: receives command-line arguments.

NOTE: Java 25 also supports compact source files and instance main methods in simplified programs, but the traditional static main signature remains standard interview knowledge.

Q2. Is Java pass-by-value?

Yes. Java is always pass-by-value. For primitives, the primitive value is copied. For objects, the value being copied is the object reference.

```
static void change(Person p) {  
    p.name = "Bob";    // Mutates the same object  
    p = new Person();  // Only changes local copy of reference  
}
```

INTERVIEW TIP: Java does not pass objects by reference. It passes a copy of the reference by value.

Q3. How many ways can a Java object be created?

A practical modern answer is four core mechanisms. Some interview lists say five by splitting reflection APIs into separate items.

- new keyword: `new Employee()`.
- Reflection: `Constructor.newInstance()` (`Class.newInstance()` is deprecated).
- Cloning: `clone()` when `Cloneable/clone` semantics are provided.
- Deserialization: reconstructing an object from serialized form.
- Factory/framework APIs may create objects, but internally they eventually use one of these mechanisms.

Q4. What is the superclass of all Java classes? Does a class extend Object plus another class?

`java.lang.Object` is the root of the Java class hierarchy. If class B extends A, and A implicitly extends `Object`, then B indirectly inherits `Object`; B is not extending two classes.

- Common `Object` methods: `equals()`, `hashCode()`, `toString()`, `getClass()`, `clone()`, `wait()`, `notify()`, `notifyAll()`.
- `finalize()` also comes from `Object` but finalization is deprecated for removal and should not be used.

```
class A { }    // A implicitly extends Object  
class B extends A { } // B -> A -> Object
```

Q5. == vs equals() vs hashCode()

- If a.equals(b) is true, a.hashCode() must equal b.hashCode().
- If hash codes are equal, objects do not have to be equal; collisions are allowed.
- If equals() is overridden, hashCode() should also be overridden.

Item	Meaning	Typical use
==	Primitive value comparison; reference identity for objects	Check same reference / primitive equality
equals()	Logical/content equality if overridden	String, value objects, domain equality
hashCode()	Integer hash used by hash-based collections	HashMap, HashSet bucket selection

INTERVIEW TIP: HashMap uses hashCode() to narrow the bucket and equals() to find the exact key within that bucket.

Q6. final vs finally vs finalize()

Keyword / Method	Meaning
final	Prevents reassignment of a variable; prevents method overriding; prevents class inheritance.
finally	Block associated with try/catch that normally runs for cleanup whether an exception occurs or not.
finalize()	Old GC finalization hook. Deprecated for removal; unpredictable and should not be used.

Q7. transient vs volatile vs static keyword

Keyword	Purpose	Key point
transient	Serialization	Field is skipped by default Java serialization.
volatile	Concurrency / visibility	Reads/writes go through JMM visibility guarantees; does not make compound operations such as count+ + atomic.
static	Class-level member	One member belongs to the class rather than each object instance.

Q8. What is Optional?

Optional<T> is a container that represents either a present non-null value or no value. It is mainly used as a return type to make absence explicit and reduce null-handling mistakes.

- Useful methods: of(), ofNullable(), empty(), map(), flatMap(), filter(), orElse(), orElseGet(), orElseThrow().
- Avoid using Optional as a field/parameter by default; it is primarily designed for return values.

```
Optional<User> user = findUser(id);  
String name = user.map(User::getName).orElse("Unknown");
```

Q9. Iterate over a String without using predefined methods

If “no predefined methods” is interpreted literally, Java provides no way to access String characters without using some String/API operation. The usual interview intent is “without toCharArray()/streams,” so use an index loop with charAt().

```
String s = "Java";  
for (int i = 0; i < s.length(); i++) {  
    System.out.println(s.charAt(i));  
}
```

2. OOP, Interfaces, Immutability & Design

[VERY IMPORTANT] Q10. Overloading vs overriding

Overloading	Overriding
Same method name, different parameter list	Subclass provides new implementation of inherited instance method
Usually compile-time polymorphism	Runtime polymorphism
Return type alone cannot overload	Covariant return type is allowed
Can occur in same class	Requires inheritance
static/final/private methods can be overloaded	final/private cannot be overridden; static is hidden

Q11. What happens when a static method is “overridden”? Method hiding

Static methods are resolved by the reference/class type, not dynamically by the runtime object. A subclass may declare a static method with the same signature, but that is method hiding, not overriding.

```
class A { static void show() { System.out.println("A"); } }
class B extends A { static void show() { System.out.println("B"); } }

A ref = new B();
ref.show(); // A
```

Q12. Overriding rules with exceptions, static and final methods

- Checked exception: overriding method may throw the same, a narrower checked exception, or none; it cannot throw a broader checked exception.
- Unchecked exception: no such narrowing restriction.
- static method: hidden, not overridden.
- final method: cannot be overridden.
- private method: not inherited, so it is not overridden.
- Access level cannot be made more restrictive in the overriding method.

Q13. Covariant return types

When overriding a method, the child method may return a subtype of the parent method's reference return type.

```
class Parent { Number getValue() { return 1; } }
class Child extends Parent {
    @Override Integer getValue() { return 1; } // Covariant return
}
```

Q14. Abstract class vs interface

Abstract class	Interface
Can have instance fields/state	Fields are implicitly public static final constants
Can have constructors	No normal constructors

Abstract class	Interface
Abstract + concrete methods	Abstract methods plus default/static methods; private helper methods are allowed in modern Java
Single class inheritance	A class can implement multiple interfaces
Use for shared state + common base behavior	Use for capability/contract across unrelated classes

INTERVIEW TIP: Choose an interface for a contract/capability; choose an abstract class when subclasses genuinely share state or implementation.

Q15. Default and static methods in interfaces

- default method: has an implementation and is inherited by implementing classes unless overridden.
- static method: belongs to the interface and is called with InterfaceName.method(); it is not inherited as an instance method.
- Java 9+ interfaces can also have private helper methods to share implementation between default/static methods.

```
interface Payment {
    default void printReceipt() { System.out.println("Receipt"); }
    static boolean valid(int amount) { return amount > 0; }
}
```

Q16. Diamond problem with two interface default methods having the same signature

The implementing class gets an ambiguous inherited default method, so it must explicitly override the method and resolve the conflict.

```
interface I1 { default void hello() { System.out.println("I1"); } }
interface I2 { default void hello() { System.out.println("I2"); } }

class Demo implements I1, I2 {
    @Override
    public void hello() {
        I1.super.hello(); // Explicit choice
    }
}
```

INTERVIEW TIP: Class methods win over interface defaults; otherwise conflicting defaults must be resolved explicitly.

Q17. Composition vs inheritance

Composition	Inheritance
HAS-A relationship	IS-A relationship
Object contains/delegates to another object	Subclass extends parent
Loose coupling, flexible behavior	Tighter coupling
Preferred for reuse when no true subtype relation exists	Use when subtype substitutability is valid

Q18. Association vs Aggregation vs Composition

Relationship	Meaning	Lifecycle
Association	General relationship between independent objects	Independent
Aggregation	Weak HAS-A ownership	Part can outlive whole
Composition	Strong HAS-A ownership	Part lifecycle tied to whole

```
// Association: Doctor <-> Patient
// Aggregation: Department has Teachers
// Composition: House has Rooms
```

Q19. What is a Singleton class?

A Singleton restricts a class to one shared instance and provides a global access point to that instance.

```
public final class Singleton {
    private Singleton() { }
    private static final Singleton INSTANCE = new Singleton();
    public static Singleton getInstance() { return INSTANCE; }
}
```

Q20. Singleton in a multithreaded environment: double-checked locking

Double-checked locking avoids synchronizing every getInstance() call after initialization. The instance must be volatile so publication is safe under the Java Memory Model.

```
public final class Singleton {
    private static volatile Singleton instance;
    private Singleton() { }

    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

INTERVIEW TIP: An enum singleton is often the simplest robust singleton in Java, especially when serialization is involved.

Q21. If a Singleton implements Serializable, what is the impact?

Deserialization normally creates a new object instance, which can break the Singleton guarantee even though the serialized object originally came from the singleton instance.

Q22. How do you resolve the Singleton serialization issue?

Implement readResolve() so deserialization returns the existing singleton instance instead of the newly reconstructed instance.

```
private Object readResolve() {
    return getInstance();
}
```

NOTE: An enum singleton gets serialization safety automatically.

[VERY IMPORTANT] Q23. Why are Strings immutable?

- String pool safety: shared literals cannot be changed by one reference and corrupt another.
- Security: Strings are used for class names, paths, URLs and other sensitive values.
- Thread safety: immutable objects can be shared safely.
- Hashing: String hash codes can be cached; Strings are reliable HashMap keys.
- Predictability: methods can safely reuse/share String instances.

Q24. How do you design an immutable class?

- Make the class final, or otherwise prevent unsafe subclassing.
- Make fields private and final.
- Initialize all state in the constructor.
- Provide no setters or mutating methods.
- Defensively copy mutable inputs.
- Return defensive copies (or immutable views) of mutable internal state.

Q25. Create an immutable Employee class containing Address as an instance variable

Because Address is mutable, Employee must make defensive copies when it receives and returns Address.

```
final class Address {
    private String city;
    Address(String city) { this.city = city; }
    String getCity() { return city; }
    void setCity(String city) { this.city = city; }
}

public final class Employee {
    private final int id;
    private final String name;
    private final Address address;

    public Employee(int id, String name, Address address) {
        this.id = id;
        this.name = name;
        this.address = new Address(address.getCity()); // Defensive copy
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public Address getAddress() {
        return new Address(address.getCity()); // Defensive copy
    }
}
```

Q26. What is a Marker interface?

A marker interface has no methods; implementing it marks a class as having a capability or meaning recognized by Java/framework code.

- Examples: Serializable, Cloneable.
- Modern designs often use annotations when behavior does not require type-system participation.

3. Strings

Q27. String pool and String immutability

String literals are interned in the String Pool. Equal literals can share one String object because String is immutable.

```
String a = "Java";  
String b = "Java";  
System.out.println(a == b); // true: same pooled literal  
  
String c = new String("Java");  
System.out.println(a == c);    // false  
System.out.println(a.equals(c)); // true
```

[VERY IMPORTANT] Q28. String vs StringBuilder vs StringBuffer

Type	Mutable?	Thread-safe?	Best use
String	No	Safe to share because immutable	Text that is not repeatedly modified
StringBuilder	Yes	No	Repeated string building in normal single-threaded code
StringBuffer	Yes	Its methods are synchronized	Shared mutable character buffer when that synchronization is specifically needed

INTERVIEW TIP: For repeated concatenation in loops, prefer StringBuilder. Simple + concatenations are fine when readability is best and the compiler can optimize them.

4. Collections, Hashing & Generics

[VERY IMPORTANT] Q29. What is the internal working of HashMap?

HashMap stores entries in an array of buckets (bins). A key's hash is spread, an index is calculated, and the entry is placed in that bin. Collisions are stored in the same bin as a linked structure and may become a red-black tree in modern Java when collision thresholds are met.

- put(): compute hash -> bucket index -> compare keys using hash + equals() -> insert/update.
- get(): compute same hash/index -> scan/tree-search the bin -> equals() confirms the key.
- Average get/put is O(1); heavy collisions can degrade performance.
- Java 8+ can treeify heavily-collided bins (commonly threshold 8, with capacity conditions).

[VERY IMPORTANT] Q30. HashMap vs synchronizedMap vs ConcurrentHashMap

HashMap	Collections.synchronizedMap(...)	ConcurrentHashMap
Not thread-safe	Thread-safe wrapper with coarse synchronization	Designed for high concurrency
Allows one null key + null values	Same null behavior as wrapped map	No null keys or values
Fail-fast iterators	Iteration requires external synchronization for a consistent traversal	Weakly consistent iterators
Best for single-threaded/external synchronization	Simple synchronized access	Best general concurrent map

Q31. How can you put all elements into the same HashMap bucket?

Create keys whose hashCode() always returns the same value while equals() still distinguishes different keys. This intentionally creates collisions.

```
final class BadKey {
    private final int id;
    BadKey(int id) { this.id = id; }

    @Override public int hashCode() { return 1; }
    @Override public boolean equals(Object o) {
        return o instanceof BadKey other && id == other.id;
    }
}
```

NOTE: This is a demonstration only. In Java 8+, a heavily-collided bin may treeify rather than remain a simple linked list.

Q32. hashCode() and equals() contract

- equals() is reflexive, symmetric, transitive, consistent, and false for null.
- If a.equals(b) is true, a.hashCode() == b.hashCode() must be true.
- Unequal objects may have the same hash code.
- Fields used by equals() should normally also be used consistently in hashCode().
- Avoid mutating key fields that participate in equals/hashCode while the object is stored as a HashMap key.

Q33. How many default buckets and what load factor does HashMap have?

The commonly cited default initial capacity is 16 and the default load factor is 0.75. The resize threshold is $\text{capacity} \times \text{loadFactor}$, so for capacity 16 the threshold is 12 entries (subject to implementation details and lazy table allocation).

Q34. Why do we need multiple buckets in HashMap?

Buckets distribute entries based on hashes so lookup can jump to a small subset of entries instead of scanning the entire map. Good distribution gives average $O(1)$ get/put performance.

Q35. How does HashSet work internally?

HashSet is backed by a HashMap. The Set element is stored as the HashMap key and all entries share a dummy PRESENT value.

```
// Conceptually inside HashSet
private transient HashMap<E, Object> map;
private static final Object PRESENT = new Object();

public boolean add(E e) {
    return map.put(e, PRESENT) == null;
}
```

INTERVIEW TIP: Uniqueness in HashSet is ultimately determined using hashCode() and equals().

Q36. Fail-Fast vs “Fail-Safe” iterators

Fail-fast	Concurrent / commonly called fail-safe
Typical ArrayList/HashMap iterators	Concurrent collections tolerate concurrent changes
May throw ConcurrentModificationException on unexpected structural modification	Usually do not throw CME just because another thread modifies the collection
Best-effort detection, not a correctness guarantee	May be snapshot-based or weakly consistent
Iterator.remove() is the supported iterator-side removal	CopyOnWriteArrayList: snapshot; ConcurrentHashMap: weakly consistent

NOTE: “Fail-safe” is common interview terminology, not an official Java Collections API classification.

Q37. What happens if I add a new element to a Map while iterating it?

With a normal HashMap iterator, adding a new key through map.put() is a structural modification and will typically cause ConcurrentModificationException on a later iterator operation. Updating only the value of an existing key is not necessarily structural. Iterator.remove() is the supported removal mechanism. ConcurrentHashMap allows concurrent modifications with a weakly consistent iterator.

```
for (Map.Entry<Integer,String> e : map.entrySet()) {
    map.put(99, "new"); // HashMap: typically ConcurrentModificationException
}
```

[VERY IMPORTANT] Q38. ConcurrentHashMap bucket-level locking

In Java 8+, ConcurrentHashMap no longer uses the Java 7 Segment architecture. It combines CAS with fine-grained synchronization on individual bins/nodes when necessary, so the entire map is not locked for normal updates.

- Reads are highly concurrent and usually do not lock.

- Empty-bin insertion can use CAS.
- Contended updates may synchronize on the bin head.
- Tree bins are used for heavy collisions under implementation thresholds.
- Java 7 used segment-level locking.

Q39. Array vs ArrayList

Array	ArrayList
Fixed length after creation	Dynamic logical size
Can store primitives directly	Stores objects; primitives use wrappers such as Integer
Lower overhead and direct indexed storage	Convenient add/remove/search APIs
Covariant and runtime type-checked	Generic and compile-time type-safe
Useful for fixed-size/performance-sensitive data	General-purpose resizable list

Q40. If an array has fixed size, why use an array instead of ArrayList?

- Lower memory overhead.
- Can store primitives without boxing.
- Good cache locality and predictable fixed size.
- Fast indexed access with minimal abstraction.
- Required by many APIs/algorithms and useful for matrices/buffers.
- If the size truly is fixed, dynamic resizing features add no value.

[VERY IMPORTANT] Q41. Comparable vs Comparator

Comparable	Comparator
Class implements Comparable<T>	Separate Comparator<T> object/function
compareTo(T)	compare(T,T)
Defines natural/default order	Defines custom/multiple orders
Requires modifying the class	Can sort third-party classes
Example: String natural order	Example: compare employees by salary, then name

```
employees.sort(Comparator.comparing(Employee::getSalary)
    .thenComparing(Employee::getName));
```

Q42. Generics and type erasure

Generics provide compile-time type safety and reusable parameterized types. Java implements most generics through type erasure: generic type parameters are removed/replaced by their bounds at runtime, with casts/bridge methods inserted as needed.

- List<String> and List<Integer> are the same raw List class at runtime.
- You cannot directly create new T() or new T[] because the concrete type is erased.
- Overloads that differ only by erased generic type can clash.

Q43. PECS: ? extends vs ? super

PECS means Producer Extends, Consumer Super.

- ? extends T: use when a structure produces T values for you to read.
- ? super T: use when a structure consumes T values you want to add.

```
List<? extends Number> src; // Read Numbers safely; do not add specific Number values  
List<? super Integer> dst; // Can add Integers; reads are Object
```

5. Functional Interfaces & Streams

Q44. How many abstract methods can a functional interface have?

Exactly one abstract method (SAM: Single Abstract Method). It may still have any number of default/static methods. Methods matching public Object methods do not count toward the single abstract method rule.

```
@FunctionalInterface
interface Calculator {
    int calculate(int a, int b);
}
```

Q45. Functional Interface vs Lambda expression

Functional Interface	Lambda
A type/contract with one abstract method	A concise implementation of that SAM
Can declare default/static methods	Has no standalone target type; it needs a compatible functional interface
Example: Predicate<T>	Example: x -> x > 0

Q46. Predicate, Supplier, Consumer, Function

Interface	Shape	Use
Predicate<T>	T -> boolean	Filtering/testing
Supplier<T>	() -> T	Produce a value
Consumer<T>	T -> void	Consume/perform side effect
Function<T,R>	T -> R	Transform one value into another

```
Predicate<Integer> even = n -> n % 2 == 0;
Supplier<String> token = () -> "abc";
Consumer<String> print = System.out::println;
Function<String,Integer> length = String::length;
```

Q47. map() vs flatMap() in Stream API

map()	flatMap()
One input element -> one transformed output element	One input -> stream/collection of outputs, then flattens
Can produce Stream<List<T>>	Can flatten to Stream<T>
Use for simple transformation	Use for nested structures

```
List<List<Integer>> data = List.of(List.of(1,2), List.of(3,4));
List<Integer> flat = data.stream()
    .flatMap(List::stream)
    .toList(); // [1,2,3,4]
```

Q48. Can we call `stream()` directly on an `int[]` array?

No instance `stream()` method exists on arrays. Use `Arrays.stream(arr)` or `IntStream.of(arr)`.

```
int[] arr = {1,2,3};  
IntStream s1 = Arrays.stream(arr);  
IntStream s2 = IntStream.of(arr);
```

6. Exceptions & Resource Management

Q49. Exception hierarchy

- Error: serious JVM/system conditions applications generally do not recover from.
- Checked exceptions: compiler enforces catch/declare.
- RuntimeException and subclasses: unchecked programming/runtime failures.

```
Object
├── Throwable
│   ├── Error
│   │   ├── OutOfMemoryError, StackOverflowError, ...
│   └── Exception
│       ├── RuntimeException // unchecked
│       └── Other Exceptions // checked
```

Q50. Checked vs unchecked exceptions

Checked	Unchecked
Subclass of Exception excluding RuntimeException	RuntimeException subclasses (and Errors are also unchecked by compiler rules)
Compiler requires catch or throws declaration	No compile-time catch/declare requirement
Example: IOException, SQLException	Example: NullPointerException, IllegalArgumentException

Q51. throw vs throws

throw	throws
Actually throws a Throwable object	Declares that a method may propagate exceptions
Used inside method/block	Used in method/constructor signature
throw new IllegalArgumentException();	void read() throws IOException

Q52. Exception handling rules with method overriding

- Overriding method cannot declare a broader checked exception than the parent method.
- It may declare the same checked exception, a subclass, or no checked exception.
- Unchecked exceptions may be added without this restriction.

```
class Parent { void read() throws IOException { } }
class Child extends Parent {
    @Override void read() throws FileNotFoundException { } // Allowed: narrower
}
```

Q53. What is try-with-resources?

try-with-resources automatically closes resources that implement `AutoCloseable`. Resources close in reverse declaration order, even when exceptions occur.

- Reduces boilerplate finally blocks.
- If both body and `close()` throw, the close exception is available as a suppressed exception.
- Since Java 9, an effectively-final existing resource can be referenced directly.

```
try (BufferedReader br = Files.newBufferedReader(path)) {  
    return br.readLine();  
}
```

Q54. `AutoCloseable` vs `Closeable`

<code>AutoCloseable</code>	<code>Closeable</code>
<code>java.lang.AutoCloseable</code>	<code>java.io.Closeable</code> extends <code>AutoCloseable</code>
<code>close()</code> may throw <code>Exception</code>	<code>close()</code> throws <code>IOException</code>
General-purpose resource contract	Primarily I/O resources
No strict idempotence requirement	<code>Closeable</code> contract expects repeated close to have no effect

7. Serialization, Copying & Object Lifecycle

Q55. Serialization vs deserialization

- serialVersionUID is used for version compatibility.
- static fields are class state and are not serialized as object state.
- transient fields are skipped by default.

Serialization	Deserialization
Object -> byte stream	Byte stream -> object
ObjectOutputStream	ObjectInputStream
Class typically implements Serializable	Reconstructs serialized object graph

Q56. Serialization with inheritance

- If a parent implements Serializable, serializable state of child objects is included automatically.
- If only the child implements Serializable and the parent does not, the non-serializable parent state is not restored from the stream; the parent no-arg constructor runs during deserialization.
- Therefore a non-serializable parent needs an accessible no-arg constructor for normal Java serialization.
- transient/static fields are not serialized as normal instance state.

Q57. Shallow copy vs deep copy

Shallow copy	Deep copy
Copies top-level object, nested references are shared	Copies the object and recursively copies mutable nested state
Changing shared child may affect both copies	Copies are independent
Faster/cheaper	More work/memory
Object.clone() is shallow by default for references	Usually implemented manually, copy constructors, mapping, or serialization-like techniques

8. Multithreading & Concurrency

Q58. Difference between wait(), sleep() and yield()

wait()	sleep()	yield()
Object method; must own monitor	Thread static method	Thread static scheduler hint
Releases the monitor while waiting	Does not release held monitors	Does not release held monitors
Waits for notify/notifyAll, interrupt or timeout	Waits for time or interrupt	Hints scheduler to let another runnable thread run
Used for coordination on a condition	Used for delay/backoff	No scheduling guarantee

Q59. Why are wait(), notify(), and notifyAll() defined in Object instead of Thread?

Because the coordination is tied to an object monitor/lock, not to the Thread object. Any Java object can act as the monitor that threads synchronize on, so the monitor wait-set operations belong to Object.

INTERVIEW TIP: wait()/notify()/notifyAll() must be called while holding that same object's monitor, normally inside synchronized(obj).

Q60. Difference between sleep(), wait() and join()

sleep()	wait()	join()
Pause current thread for time	Wait on an object condition	Wait for another thread to finish
Thread.sleep	Object.wait	Thread.join
Does not release monitor	Releases the monitor being waited on	Caller waits until target thread terminates or timeout/interruption
No synchronized block required	Must own object monitor	No monitor ownership requirement

Q61. What is ThreadLocal?

ThreadLocal gives each thread its own independent value associated with the same ThreadLocal variable. It is useful for per-thread context when passing a parameter everywhere would be awkward.

```
private static final ThreadLocal<String> REQUEST_ID = new ThreadLocal<>();
try {
    REQUEST_ID.set("req-123");
    // use REQUEST_ID.get()
} finally {
    REQUEST_ID.remove(); // Important for pooled threads
}
```

NOTE: With thread pools, forgetting remove() can retain stale data and contribute to memory leaks.

[VERY IMPORTANT] Q62. volatile vs Atomic classes

volatile	AtomicInteger / AtomicReference etc.
Visibility + ordering guarantee for a variable	Atomic read-modify-write operations
Read/write of the variable is visible across threads	Methods such as incrementAndGet(), compareAndSet()

volatile	AtomicInteger / AtomicReference etc.
count++ is NOT made atomic	count.incrementAndGet() is atomic
Good for flags/state publication	Good for lock-free counters/state transitions

Q63. CountdownLatch vs CyclicBarrier

CountDownLatch	CyclicBarrier
One or more threads wait until count reaches zero	A fixed number of threads wait for each other at a barrier
countDown() + await()	await()
One-shot; cannot reset	Reusable after barrier trips
Good for “wait until N tasks finish”	Good for phased parallel work

[VERY IMPORTANT] Q64. Future vs CompletableFuture

Future	CompletableFuture
Represents one asynchronous result	Future plus fluent asynchronous composition
get() commonly blocks	thenApply/thenCompose/thenCombine/allOf enable pipelines
Limited chaining/error handling	Rich exception handling and callbacks
Usually obtained from ExecutorService.submit()	Can be created with runAsync/supplyAsync or completed manually

Q65. CompletableFuture: runAsync() vs supplyAsync()

runAsync()	supplyAsync()
Runs Runnable	Runs Supplier<T>
No result value	Produces a result
CompletableFuture<Void>	CompletableFuture<T>

```
CompletableFuture<Void> a = CompletableFuture.runAsync() -> doWork();
CompletableFuture<String> b = CompletableFuture.supplyAsync() -> loadData();

b.thenApply(String::toUpperCase)
  .exceptionally(ex -> "fallback");
```

Q66. Object-level lock vs class-level lock

Object-level lock	Class-level lock
Locks one instance monitor	Locks the Class object monitor

Object-level lock	Class-level lock
synchronized instance method / synchronized(this)	static synchronized / synchronized(MyClass.class)
Different instances can execute concurrently	All instances share the same class lock for that critical section

Q67. What is ReentrantLock?

ReentrantLock is an explicit lock from java.util.concurrent.locks. “Reentrant” means the same thread can acquire the same lock multiple times.

- Supports tryLock(), lockInterruptibly(), optional fairness and multiple Condition objects.
- Always unlock in finally.

```
lock.lock();
try {
    // critical section
} finally {
    lock.unlock();
}
```

Q68. How do you achieve thread safety in a multithreaded environment?

- Prefer immutability and avoid shared mutable state.
- Use thread confinement / local variables where possible.
- Use synchronized or explicit Lock for critical sections.
- Use atomics for simple atomic state transitions/counters.
- Use concurrent collections such as ConcurrentHashMap and BlockingQueue.
- Use safe publication (final fields, volatile, synchronization, concurrent containers).
- Use higher-level executors/futures instead of manual thread coordination when possible.

Q69. Explain ExecutorService and types of executors

ExecutorService separates task submission from thread management. You submit Runnable/Callable tasks and the executor manages worker threads and lifecycle.

- Call shutdown() for orderly shutdown; awaitTermination() if needed.

Factory / Executor	Typical use
newFixedThreadPool(n)	Bounded number of reusable workers
newCachedThreadPool()	Creates/reuses threads as needed; can grow aggressively
newSingleThreadExecutor()	Sequential task execution
newScheduledThreadPool(n)	Delayed/periodic tasks
newWorkStealingPool()	ForkJoinPool-based work stealing
newVirtualThreadPerTaskExecutor() (Java 21+)	One virtual thread per submitted task; useful for high-concurrency blocking I/O

Q70. `newCachedThreadPool()` vs `newFixedThreadPool()`

Cached	Fixed
Potentially creates many threads when load grows	At most N worker threads
Uses idle thread reuse	Tasks wait in queue when all workers busy
Good for many short-lived tasks only when growth is controlled	Predictable concurrency/resource limit
Risk: unbounded thread growth under sustained load	Risk: queue growth if producers outpace workers

Q71. Explain deadlock

A deadlock occurs when threads are permanently waiting for locks/resources held by each other.

- Classic conditions: mutual exclusion, hold-and-wait, no preemption, circular wait.
- Prevention: consistent lock ordering, reduce nested locking, use `tryLock`/timeouts, minimize critical sections.
- Diagnosis: thread dumps (`jstack/jcmd`), JFR, profilers.

```
// Risk: opposite lock order
Thread 1: lock(A) -> lock(B)
Thread 2: lock(B) -> lock(A)
```

Q72. `ForkJoinPool` and work stealing

`ForkJoinPool` is designed for recursive/parallel tasks. Work is split into subtasks; each worker has a deque. An idle worker can steal tasks from another worker's deque, improving CPU utilization.

- `RecursiveTask<V>`: returns a result.
- `RecursiveAction`: no result.
- Best suited to CPU-bound divide-and-conquer work; avoid blocking tasks unless designed appropriately.

Q73. Write a program where one thread prints odd numbers and another prints even numbers

```
class OddEvenPrinter {
    private int n = 1;
    private final int max;
    OddEvenPrinter(int max) { this.max = max; }

    synchronized void printOdd() throws InterruptedException {
        while (n <= max) {
            while (n <= max && n % 2 == 0) wait();
            if (n <= max) System.out.println("Odd : " + n++);
            notifyAll();
        }
    }

    synchronized void printEven() throws InterruptedException {
        while (n <= max) {
            while (n <= max && n % 2 != 0) wait();
            if (n <= max) System.out.println("Even : " + n++);
            notifyAll();
        }
    }

    public static void main(String[] args) {
        OddEvenPrinter p = new OddEvenPrinter(10);
        new Thread() -> { try { p.printOdd(); } catch (InterruptedException e) { Thread.currentThread().interrupt(); } }).start();
        new Thread() -> { try { p.printEven(); } catch (InterruptedException e) { Thread.currentThread().interrupt(); } }).start();
    }
}
```

Q74. Implement a bounded blocking queue using wait() and notifyAll()

```
class SimpleBlockingQueue<T> {
    private final Queue<T> q = new ArrayDeque<>();
    private final int capacity;
    SimpleBlockingQueue(int capacity) { this.capacity = capacity; }

    public synchronized void put(T item) throws InterruptedException {
        while (q.size() == capacity) wait();
        q.add(item);
        notifyAll();
    }

    public synchronized T take() throws InterruptedException {
        while (q.isEmpty()) wait();
        T item = q.remove();
        notifyAll();
        return item;
    }
}
```

INTERVIEW TIP: Use while, not if, around wait() because of spurious wakeups and because another thread may consume the condition before you reacquire the lock. In production, prefer `java.util.concurrent.BlockingQueue`.

Q75. Write a Java program to print a sequence up to N using 3 threads in order

```
class ThreeThreadSequence {
    private int n = 1;
    private final int max;
    ThreeThreadSequence(int max) { this.max = max; }

    synchronized void print(int turn) throws InterruptedException {
        while (n <= max) {
            while (n <= max && (n - 1) % 3 != turn) wait();
            if (n <= max) {
                System.out.println("THREAD-" + (turn + 1) + " : " + n++);
                notifyAll();
            }
        }
    }
}

public static void main(String[] args) {
    ThreeThreadSequence p = new ThreeThreadSequence(10);
    for (int i = 0; i < 3; i++) {
        int turn = i;
        new Thread() -> {
            try { p.print(turn); }
            catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        }.start();
    }
}
```

9. JVM Memory Model, OOM & Garbage Collection

Q76. What is the Java Memory Model (JMM)?

The JMM defines the rules for how threads interact through memory: visibility, ordering, and atomicity of reads/writes. It explains when one thread is guaranteed to see another thread's actions.

- happens-before relationships provide visibility/order guarantees.
- synchronized establishes lock/unlock happens-before relationships.
- volatile write happens-before a later volatile read of the same variable.
- Thread start/join and final-field rules also provide guarantees.

NOTE: JMM is not the same thing as JVM runtime memory areas such as heap, stack, metaspace and native memory.

Q77. What can cause OutOfMemoryError and how do you fix it?

Diagnosis/fix: capture the exact OOM message, heap dump and GC/JFR data; identify dominant retainers; fix leaks/unbounded caches or right-size memory only after understanding the cause.

- Java heap exhausted: true workload pressure or retained-object leak.
- Metaspace exhaustion: excessive class loading/classloader leaks.
- Direct/native memory exhaustion: direct buffers/native allocations.
- Unable to create native thread: too many threads or OS/native-memory limits.
- GC overhead limit: JVM spends excessive time collecting with little reclaimed memory.

Useful options/tools:

-XX:+HeapDumpOnOutOfMemoryError

jcmd / jmap / jstack / JFR

VisualVM (the question bank says "VirtualVM")

JProfiler

Q78. What is a memory leak in Java and how do you prevent it?

A Java memory leak means objects are no longer useful to the application but are still strongly reachable, so GC cannot reclaim them.

- Bound caches/collections; avoid ever-growing static maps.
- Close resources with try-with-resources.
- Remove listeners/callback registrations when no longer needed.
- Use ThreadLocal.remove() in pooled-thread scenarios.
- Avoid accidental classloader retention.
- Profile heap dominator/retained-size paths with JProfiler, VisualVM, JFR or heap analyzers.

Q79. Explain mark-sweep garbage collection algorithm

- Mark: start from GC roots and mark reachable objects.
- Sweep: reclaim memory occupied by unmarked/unreachable objects.
- Problem: sweeping alone can leave fragmentation.
- Mark-compact adds compaction so live objects are moved together.

[VERY IMPORTANT] Q80. CMS vs G1 vs ZGC

CMS	G1	ZGC
Legacy concurrent mark-sweep collector; removed from modern JDKs	Region-based collector; default in mainstream HotSpot since Java 9	Low-latency collector designed for very large heaps and very short pauses
Focused on reducing old-gen pauses but suffered fragmentation	Balances throughput and pause-time goals; performs young and mixed collections	Performs most work concurrently; modern ZGC supports generational operation
Do not choose for modern Java because it is removed	Strong general-purpose default	Choose when tail latency/very short pauses are a primary requirement

Q81. GC types/use cases and Eden memory

Generational collectors exploit the observation that most objects die young. New allocations generally start in Eden (young generation), surviving objects may move through survivor regions/spaces and eventually be promoted to old generation.

- Young/Minor GC: collects young-generation objects.
- Mixed GC (G1): can collect young regions plus selected old regions.
- Full GC: broader stop-the-world collection; usually more expensive and something to investigate if frequent.
- Eden is where most new objects are allocated, often using thread-local allocation buffers (TLABs).
- G1 uses regions but still has logical Eden/Survivor/Old roles.

Q82. Java garbage collectors: when would you use which?

Collector	Typical choice
Serial GC	Small heaps/simple applications where one GC thread is sufficient
Parallel GC	Throughput-oriented batch/CPU workloads where pauses are acceptable
G1	General server workloads needing balanced throughput and pause goals
ZGC	Large heaps and latency-sensitive systems requiring very small pauses

10. Modern Java: 8 / 11 / 17 / 21 / 25

Q83. Top 2 important features of Java 8, 11, 17, 21 and 25

Version	Two interview-relevant features
Java 8	Lambda expressions + functional interfaces; Stream API (plus Optional/default interface methods)
Java 11	Standard HTTP Client; useful String/Files API additions such as <code>isBlank()</code> , <code>lines()</code> , <code>repeat()</code> , <code>readString()</code> / <code>writeString()</code>
Java 17	Sealed classes; pattern matching for instanceof. Records are also widely used in Java 17 although finalized in Java 16.
Java 21	Virtual threads; pattern matching for switch + record patterns
Java 25	Scoped Values finalized; Module Import Declarations finalized. Compact Source Files + Instance Main Methods are also a Java 25 language feature.

NOTE: Java feature/version answers should distinguish “introduced/finalized in version X” from “commonly available in an LTS version.”

Q84. What is a Java record? Are records immutable?

A record is a compact class for transparent data carriers. The compiler generates component accessors, `equals()`, `hashCode()`, `toString()` and a canonical constructor structure.

- Record components are final fields, so the record is shallowly immutable.
- If a component refers to a mutable object such as `List`, that object can still mutate unless you defensively copy it.
- Records are implicitly final and can implement interfaces.

```
record Employee(int id, String name) { }
```

Q85. What is a sealed class?

A sealed class/interface restricts which classes/interfaces may directly extend or implement it.

- Permitted subclasses must explicitly be final, sealed, or non-sealed (subject to the language rules).
- Useful for controlled hierarchies and exhaustive pattern matching.

```
sealed interface Shape permits Circle, Rectangle { }  
final class Circle implements Shape { }  
non-sealed class Rectangle implements Shape { }
```

Q86. What is pattern matching in modern Java?

Pattern matching reduces boilerplate when checking a type and extracting its value. Modern Java includes pattern matching for instanceof, switch and record patterns.

```
if (obj instanceof String s) {  
    System.out.println(s.length());  
}  
  
String result = switch (shape) {  
    case Circle c -> "circle";  
    case Rectangle r -> "rectangle";  
};
```

[VERY IMPORTANT] Q87. What is a virtual thread?

A virtual thread is a lightweight Thread managed by the JVM rather than being permanently mapped one-to-one to an OS thread. Java 21 finalized virtual threads.

- Excellent for high-concurrency workloads that spend much time blocking on I/O.
- They simplify the thread-per-request style without needing huge platform-thread pools.
- They do not make CPU-bound work faster; CPU parallelism is still limited by cores.

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {  
    executor.submit(() -> callRemoteService());  
}
```

11. Annotations

Q88. What are annotations? Can you create your own annotations?

Annotations attach metadata to code elements. The compiler, runtime reflection, annotation processors and frameworks can inspect that metadata. Yes, you create a custom annotation using `@interface`.

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Audit {
    String value() default "";
}
```

Q89. What built-in annotations are provided by Java?

- `@Override` - verifies a method overrides/implements a supertype method.
- `@Deprecated` - marks APIs that should no longer be used.
- `@SuppressWarnings` - suppresses selected compiler warnings.
- `@SafeVarargs` - asserts certain generic varargs usage is safe.
- `@FunctionalInterface` - verifies a SAM/functional interface.
- Meta-annotations include `@Retention`, `@Target`, `@Documented`, `@Inherited` and `@Repeatable`.

12. Testing, Mockito & Dependency Injection

Q90. Difference between Unit Testing and Integration Testing

Unit test	Integration test
Tests one unit/class/function in isolation	Tests interaction between multiple components/systems
Dependencies often mocked/stubbed	Uses more real dependencies/infrastructure
Very fast, deterministic, many tests	Slower, fewer tests, catches wiring/contract issues
Example: service method with mocked repository	Example: service + real DB/test container + HTTP layer

Q91. How do you assert that code throws an exception in JUnit 5?

```
@Test
void rejectsNegativeAge() {
    IllegalArgumentException ex = assertThrows(
        IllegalArgumentException.class,
        () -> service.setAge(-1)
    );
    assertEquals("age must be positive", ex.getMessage());
}
```

Q92. @Mock vs @InjectMocks in Mockito

@Mock	@InjectMocks
Creates a Mockito mock for a dependency	Creates the class under test and tries to inject available mocks/spies into it
No real behavior unless stubbed	The object itself is generally real; its dependencies may be mocks
Example: Repository repository	Example: Service service receiving repository

Q93. Mock vs Spy

Mock	Spy
Fully simulated object	Wraps/uses a real object by default
Methods return Mockito defaults unless stubbed	Real methods execute unless stubbed
Best for dependencies you want isolated	Useful for partial mocking, but use sparingly
when(mock.m()).thenReturn(x)	doReturn(x).when(spy).m() is often safer

Q94. doReturn() vs thenReturn()

Both can stub non-void methods, but the important difference appears with spies. `when(spy.realMethod()).thenReturn(x)` calls the real method during stubbing; `doReturn(x).when(spy).realMethod()` does not.

```
doReturn("stubbed").when(spy).load();
when(mock.load()).thenReturn("stubbed");
```

Q95. How do you mock a void method?

- Void methods are commonly tested by verifying interaction or state change rather than returning a value.

```
doNothing().when(mock).save(any());
doThrow(new RuntimeException("boom")).when(mock).delete(any());
doAnswer(inv -> { /* custom behavior */ return null; }).when(mock).send(any());

verify(mock).save(any());
```

Q96. Constructor injection vs setter injection

Constructor injection	Setter injection
Best for required dependencies	Useful for optional/reconfigurable dependencies
Object is valid immediately after construction	Object may exist temporarily incomplete
Supports final fields/immutability	Dependencies usually remain mutable
Easy to test explicitly	Can hide required dependencies

INTERVIEW TIP: For required dependencies, constructor injection is generally preferred.

13. Reactive Programming

Q97. Reactive programming: Mono and Flux with non-blocking I/O

Project Reactor is a reactive library commonly used by Spring WebFlux. `Mono<T>` represents 0 or 1 asynchronous element; `Flux<T>` represents 0 to many elements.

- Reactive streams support asynchronous pipelines and backpressure.
- They work well with non-blocking I/O when the entire call chain is non-blocking.
- Putting a blocking JDBC/file/network call inside a `Mono/Flux` does not magically make that operation non-blocking.
- Common operators: `map`, `flatMap`, `filter`, `zip`, `onErrorResume`, `subscribe`.

```
Mono<User> oneUser = userClient.findById(id);  
Flux<User> manyUsers = userClient.findAll();
```

14. Stream API Coding Questions

The original question bank already contained several answers. They are reorganized here and normalized into interview-ready Java code.

Q98. Count every character in a String using Stream API

```
String str = "abcdabcbghsk";
Map<Character, Long> counts = str.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
System.out.println(counts);
```

Q99. Merge two lists using Stream API

```
List<Integer> list1 = Arrays.asList(1, 2, 3, 4);
List<Integer> list2 = Arrays.asList(5, 6, 7, 8);

List<Integer> merged = Stream.concat(list1.stream(), list2.stream()).toList();
// Alternative: Stream.of(list1, list2).flatMap(List::stream).toList();
```

Q100. Reverse a String using Stream API

```
String str = "Stream";
String reversed = IntStream.range(0, str.length())
    .mapToObj(i -> String.valueOf(str.charAt(str.length() - 1 - i)))
    .collect(Collectors.joining());
System.out.println(reversed); // maertS
```

Q101. Delete all occurrences of 'c' from a String

```
String s = "occurance";
String ans = s.chars()
    .mapToObj(c -> (char) c)
    .filter(ch -> ch != 'c')
    .map(String::valueOf)
    .collect(Collectors.joining());
System.out.println(ans); // ourane
```

Q102. List duplicate characters in a String

```
String s = "programming";
List<Character> duplicates = s.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
    .entrySet().stream()
    .filter(e -> e.getValue() > 1)
    .map(Map.Entry::getKey)
    .toList();
System.out.println(duplicates);
```

Q103. Count how many times each word appears in a List<String>

```
List<String> words = Arrays.asList("apple", "banana", "apple", "orange", "banana");
Map<String, Long> wordCount = words.stream()
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
// apple=2, banana=2, orange=1
```

Q104. Return the second-highest distinct word length in a sentence

```
String sentence = "I am working in motivated organization";
int result = Arrays.stream(sentence.split("\\s+"))
    .map(String::length)
    .distinct()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst()
    .orElse(0);
System.out.println(result); // 9
```

Q105. Find even integers from a mixed List<String>

```
List<String> list = Arrays.asList("one", "5", "ab12", "abc", "8", "20");
List<Integer> even = list.stream()
    .filter(Objects::nonNull)
    .filter(s -> s.matches("-?\\d+"))
    .map(Integer::parseInt)
    .filter(n -> n % 2 == 0)
    .toList();
System.out.println(even); // [8, 20]
```

Q106. Names ending with "lal" using Stream API

```
List<String> list = Arrays.asList("Ram", "Vinitha", "Mohanlal", "Jawaharlal");
List<String> result = list.stream()
    .filter(name -> name.endsWith("lal"))
    .toList();
```

Q107. Names NOT ending with "lal" using Stream API

```
List<String> result = list.stream()
    .filter(name -> !name.endsWith("lal"))
    .toList();
```

Q108. If the names list contains null, avoid NullPointerException and return names not ending with "lal"

```
List<String> result = list.stream()
    .filter(Objects::nonNull)
    .filter(name -> !name.endsWith("lal"))
    .toList();
```

Q109. Find the second-largest number using Stream API

```
List<Integer> numbers = Arrays.asList(10, 5, 15, 20, 8);
Integer secondLargest = numbers.stream()
    .distinct()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst()
    .orElse(null); // 15
```

Q110. Find unique characters of a String using Stream API

```
String input = "hello world";
Set<Character> uniqueChars = input.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.toSet());
```

NOTE: Collectors.toSet() does not guarantee encounter order. Use LinkedHashSet::new if insertion order matters.

Q111. Convert List<Employee> to Map<empId, empName>

```
Map<Integer, String> employeeMap = employees.stream()
    .collect(Collectors.toMap(Employee::getEmpId, Employee::getEmpName));
```

NOTE: If duplicate empId values are possible, provide a merge function to avoid IllegalStateException.

Q112. Convert List<Employee> to Map<empId, empName> while maintaining insertion order

```
Map<Integer, String> employeeMap = employees.stream()
    .collect(Collectors.toMap(
        Employee::getEmpId,
        Employee::getEmpName,
        (v1, v2) -> v1,
        LinkedHashMap::new
    ));
```

Q113. Find the vehicle that travelled the maximum total distance in August

```
String maxDistanceVehicle = journeyList.stream()
    .filter(j -> j.getDateOfJourney().getMonthValue() == 8)
    .collect(Collectors.groupingBy(
        Journey::getVehicleId,
        Collectors.summingDouble(Journey::getDistanceTravelled)))
    .entrySet().stream()
    .max(Map.Entry.comparingByValue())
    .flatMap(e -> vehicleList.stream()
        .filter(v -> Objects.equals(v.getId(), e.getKey()))
        .findFirst())
    .map(Vehicle::getName)
    .orElse("No vehicles travelled in August");
```

NOTE: If multiple years are present, also filter by year; “August” alone combines all August journeys across years.

Q114. Return duplicated integers from an ArrayList<Integer>

```
List<Integer> nums = Arrays.asList(1, 2, 3, 2, 4, 5, 3, 6);
List<Integer> duplicates = nums.stream()
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
    .entrySet().stream()
    .filter(e -> e.getValue() > 1)
    .map(Map.Entry::getKey)
    .toList(); // [2, 3] (order depends on map implementation)
```

NOTE: This avoids nested streams and avoids relying on Integer reference comparison with ==.

Q115. Return employees whose name starts with 'A' and city is Mumbai or Pune

```
Predicate<Employee> nameStartsWithA = e -> e.getName().startsWith("A");
Predicate<Employee> cityIsMumbaiOrPune = e ->
    e.getCity().equalsIgnoreCase("Mumbai") || e.getCity().equalsIgnoreCase("Pune");

List<Employee> result = employees.stream()
    .filter(nameStartsWithA.and(cityIsMumbaiOrPune))
    .toList();
```

Final 2-Minute Revision Sheet

Topic	One-line answer
HashMap	hash -> bucket -> equals; default capacity 16, load factor 0.75; collisions may treeify.
ConcurrentHashMap	Java 8+ CAS + fine-grained bin synchronization; no null keys/values; weakly consistent iterators.
Comparable vs Comparator	compareTo = natural order; compare = external/custom order.
String / Builder / Buffer	String immutable; Builder mutable fastest; Buffer mutable synchronized.
Functional interface	Exactly one abstract method; lambda supplies its implementation.
PECS	Producer Extends, Consumer Super.
Exception override	Cannot broaden checked exception.
volatile vs Atomic	volatile visibility/order; Atomic gives atomic compound operations via CAS-style APIs.
wait vs sleep	wait releases monitor and needs ownership; sleep does not release monitor.
Future vs CompletableFuture	Future is basic result; CompletableFuture supports composition and callbacks.
JMM	Visibility + ordering + atomicity rules via happens-before.
GC	G1 general default; ZGC low latency; CMS is removed.
Record	Compact data carrier; shallowly immutable.
Virtual thread	Lightweight Java 21 thread, excellent for high-concurrency blocking I/O.
Mockito spy	Real methods by default; doReturn is safer when stubbing a spy.